DevOps / Site Reliability Engineer
Interview Preparation Guide

Technical interview questions • Sample answers • Employer screening questions
PostgreSQL fundamentals and operational reviewer

Tailored to a cloud-native, Azure-based multi-tenant SaaS role requiring CI/CD, Infrastructure as Code, containers, PostgreSQL, monitoring, high availability, and disaster recovery.

Personalization note

This guide uses the experience previously provided: six years in DevOps, AWS as the main cloud platform, and experience with Azure Cloud, Azure DevOps, Terraform, Docker, GitHub Enterprise, GitHub Advanced Security, and SonarQube. Keep all interview answers truthful to your actual hands-on experience.

Study strategy

- Prioritize Azure architecture and high availability.
- Practice PostgreSQL SQL, query troubleshooting, backup, and recovery.
- Prepare real examples of CI/CD improvements and incident resolution.
- Be clear about the difference between DevOps experience and dedicated SRE experience.

Contents and study roadmap

Section	Topic
1	Introduction and general interview questions
2	Azure cloud infrastructure and high availability
3	CI/CD, Terraform, Docker, and Kubernetes
4	Monitoring, observability, and incident management
5	Multi-tenant SaaS architecture and security
6	Scenario-based interview questions
7	PostgreSQL fundamentals and basics
8	Employer application questions
9	Behavioral interview questions
10	Questions to ask the interviewer and study plan

Suggested priority: PostgreSQL production support, Azure HA/DR, GitHub Actions and Terraform, SRE metrics, and multi-tenant security.

1. Introduction and general interview questions

1. Tell me about yourself.

Sample answer

I’m a DevOps Engineer with six years of experience working with cloud infrastructure, CI/CD pipelines, infrastructure automation, and application delivery. My primary cloud platform is AWS, and I also have experience with Azure Cloud and Azure DevOps, along with Terraform, Docker, GitHub Enterprise, GitHub Advanced Security, and SonarQube. My focus is on improving deployment consistency, automating repetitive tasks, strengthening security practices, and making infrastructure more reliable and maintainable. I’m interested in this opportunity because it combines cloud engineering with site reliability, disaster recovery, database operations, and a business-critical SaaS platform. I’m looking to contribute my existing DevOps experience while deepening my expertise in Azure, PostgreSQL, and production reliability engineering.

2. Why are you interested in this position?

Sample answer

I’m interested because this role goes beyond maintaining pipelines. It involves the complete reliability lifecycle, from infrastructure design and automated deployments to monitoring, disaster recovery, and incident response. Those responsibilities align with my DevOps background and the direction I want to grow in. I’m particularly interested in strengthening my Azure platform engineering and PostgreSQL operations skills while helping the team improve availability, deployment safety, and operational efficiency.

3. Your main cloud platform is AWS. How will you adapt to an Azure-focused environment?

Sample answer

My strongest cloud experience is in AWS, but the underlying cloud engineering principles are transferable. High availability, identity and access management, networking, infrastructure as code, monitoring, automation, and disaster recovery follow similar principles across cloud providers. I have experience with Azure Cloud and Azure DevOps, so I would build on that foundation and deepen my knowledge of services such as Azure Virtual Network, Azure Monitor, Azure Key Vault, Azure Kubernetes Service, and Azure Database for PostgreSQL. I would review the existing architecture, deployment standards, security controls, and recovery procedures before making changes.

4. What is the difference between DevOps and SRE?

Sample answer

DevOps is a set of practices and cultural principles that bring development and operations closer together through collaboration, automation, and continuous delivery. SRE applies software engineering and measurable reliability practices to operations, using service-level indicators, service-level objectives, monitoring, automation, incident management, and error budgets. They overlap significantly: DevOps practices help automate delivery and infrastructure, while SRE practices help measure service health, reduce incidents, and improve availability.

5. What are your strengths as a DevOps Engineer?

Sample answer

My strengths are infrastructure automation, CI/CD, troubleshooting, and thinking about repeatability and operational risk. I look for ways to reduce manual work, make deployments consistent, and integrate security and quality checks into delivery pipelines. I also approach incidents methodically by assessing impact, gathering evidence, restoring service safely, and identifying preventive improvements afterward.

Interview tip: Prepare a real example of each strength and be ready to explain your personal contribution.

2. Azure cloud infrastructure and high availability

6. How would you design a highly available Azure architecture for a multi-tenant SaaS application?

Sample answer

I would start by identifying availability requirements, traffic patterns, security boundaries, and recovery objectives. I would evaluate Azure Front Door or an appropriate application entry point for routing, and Application Gateway where application-layer routing or WAF capabilities are needed. I would distribute application instances across availability zones where supported. For containers, I would evaluate AKS or Azure Container Apps based on complexity. I would configure Azure Database for PostgreSQL for appropriate high availability and backups, use Key Vault for secrets, and Azure Monitor and Application Insights for observability. I would automate deployments and infrastructure, test recovery procedures, and enforce tenant isolation at the application and data layers.

Reference architecture (illustrative)

Layer	Example responsibilities / services
Entry and routing	Azure Front Door and/or Application Gateway; TLS, routing, optional WAF
Application	Multiple React frontend and .NET API instances; zones and health checks where supported
Database	Azure Database for PostgreSQL; HA configuration, backups, recovery, restricted access
Platform controls	Terraform, GitHub Actions, Key Vault, Azure Monitor, alerts, DR runbooks

This is a reference design, not a final architecture. The right services depend on the existing application, service-level objectives, region availability, budget, and compliance requirements.

7. What is high availability?

Sample answer

High availability means designing a system to remain operational despite individual component failures. It involves redundancy, health checks, automatic failover, load balancing, and removal of single points of failure. I would distribute application instances across availability zones where supported, configure appropriate health probes, ensure database availability, monitor critical dependencies, and test failover.

8. What is the difference between availability zones and disaster recovery?

Sample answer

Availability zones are physically separate locations within an Azure region designed to reduce the impact of localized failures. Disaster recovery focuses on restoring service after a larger failure, such as a regional outage or major data corruption. Zone redundancy helps with certain infrastructure failures, but it does not replace cross-region DR, backups, or recovery testing.

9. What are RTO and RPO?

Sample answer

RTO (Recovery Time Objective) is the maximum acceptable time to restore a service after a disruption. RPO (Recovery Point Objective) is the maximum acceptable data loss measured in time. For example, an RTO of one hour means the service should be restored within one hour; an RPO of 15 minutes means recovery should lose no more than approximately 15 minutes of data. I would agree these objectives with the business and validate them through recovery exercises.

10. How would you implement disaster recovery for an Azure SaaS application?

Sample answer

I would identify critical services and dependencies, define RTO/RPO, configure database backups and point-in-time recovery, assess cross-region needs, maintain version-controlled infrastructure definitions, and document how to restore application services, networking, secrets, and dependencies. I would automate recovery steps where practical and regularly test restoration and failover. After each test, I would record actual recovery times, data recovery points, failures, and corrective actions.

11. What is the difference between backup and replication?

Sample answer

Backup provides a recoverable copy of data at a point in time or across a retention window. Replication maintains another copy of data on a separate system, often for availability or DR. Replication can also copy accidental deletion or corruption, while backups provide additional protection through retention and deletion controls. A reliable strategy uses both where appropriate and tests restoration.

12. How would you improve infrastructure performance and scalability?

Sample answer

I would establish a baseline for CPU, memory, latency, request throughput, database performance, and error rates. Then I would identify the bottleneck rather than scaling everything blindly. I might tune queries, optimize connection pooling, introduce caching, scale application instances horizontally, adjust resource limits, or improve networking. I would validate changes with load tests and production metrics while monitoring cost and security.

3. CI/CD, Terraform, Docker, and Kubernetes

13. How would you design a secure CI/CD pipeline using GitHub Actions?

Sample answer

I would structure the pipeline into validation, build, testing, security scanning, artifact publishing, deployment, and post-deployment verification. For .NET, I would run unit tests, code-quality checks, dependency scanning, and build the application. If needed, I would build and scan a container image before publishing it to a controlled registry. I would use environment protections, least-privilege permissions, and short-lived Azure credentials through OpenID Connect where supported. Production deployments would use approvals, controlled rollout, health checks, and rollback or mitigation if a release failed.

14. What is Infrastructure as Code, and why use Terraform?

Sample answer

Infrastructure as Code manages infrastructure through version-controlled definitions rather than manual configuration. Terraform lets us define resources declaratively, review changes through plans, reuse modules, and maintain consistency. I would use reusable modules, secure remote state with locking where supported, environment-specific configuration, code review, and careful handling of state files.

15. What is Terraform state, and why is it important?

Sample answer

Terraform state maps configuration to the actual resources Terraform manages. It helps Terraform understand existing resources and determine required changes. For a team, I would use a secure remote backend, state locking where supported, restricted access, versioning and recovery protections, and avoid storing secrets in state wherever possible. I would not casually edit or delete production state.

16. What is the difference between terraform plan and terraform apply?

Sample answer

`terraform plan` previews the changes Terraform intends to make. `terraform apply` executes those changes. My normal workflow is to format and validate configuration, review a plan, obtain approval, and apply the reviewed changes. Production infrastructure changes should be controlled and reviewed.

17. How would you deploy an application with minimal downtime?

Sample answer

I would choose a strategy based on architecture and risk: rolling deployment replaces instances gradually; blue-green maintains two environments and switches traffic after validation; canary releases to a small portion of traffic before expanding. I would include health checks, backward-compatible database migrations, smoke tests, monitoring, and a rollback or mitigation plan. I would verify that old and new application versions can coexist during a release.

18. What is the difference between Docker and Kubernetes?

Sample answer

Docker is commonly used to build and run container images. Kubernetes orchestrates containers across a cluster, handling scheduling, desired replica counts, service discovery, rolling updates, and recovery from certain workload failures. For Azure, AKS provides a managed Kubernetes control plane, while the team still manages workload configuration, security, sizing, networking, and operations.

19. How would you troubleshoot a Kubernetes pod that keeps restarting?

Sample answer

I would inspect the pod status and events with `kubectl describe pod`, then review current and previous container logs. I would check application crashes, missing configuration or secrets, failed probes, insufficient memory, CPU constraints, and dependency failures. If it is an OOM termination, I would investigate memory use and limits. I would make the smallest safe change, verify recovery, and add preventive monitoring.

```
kubectl get pods -n <namespace>
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace>
kubectl logs <pod-name> --previous -n <namespace>
kubectl get events -n <namespace> --sort-by=.lastTimestamp
kubectl top pods -n <namespace>
```

20. How do you manage secrets in CI/CD pipelines?

Sample answer

I would not commit credentials to source control or embed them in container images. I would use a secrets manager such as Azure Key Vault and access it through a managed identity or configured federated identity. I would apply least privilege, restrict production access, rotate credentials when needed, prevent secret values from appearing in logs, scan for accidental exposure, and follow a revocation procedure if a credential leaked.

4. Monitoring, observability, and incident management

21. What is the difference between monitoring and observability?

Sample answer

Monitoring shows whether known conditions are healthy or unhealthy through metrics, dashboards, and alerts. Observability helps explain why a system behaves a certain way by correlating metrics, logs, and traces. I would monitor request latency, error rates, traffic, resource utilization, database connections, and failed deployments. Distributed tracing can help investigate slow requests across React, .NET services, and PostgreSQL.

22. What are SLIs, SLOs, and SLAs?

Sample answer

An SLI is a measured aspect of service health, such as successful request percentage. An SLO is the target for that indicator, such as 99.9% successful requests. An SLA is a formal commitment to customers, potentially with remedies. I would define SLIs around customer experience, establish SLOs, and use error budgets to inform reliability and deployment decisions.

Term	Meaning	Example
SLI	Measured service indicator	Successful request percentage
SLO	Target for an indicator	99.9% success target
SLA	Customer-facing commitment	Contractual uptime commitment

23. How would you investigate a sudden increase in application latency?

Sample answer

I would establish scope and impact: whether all tenants are affected, whether latency increased after a deployment, and whether it is isolated to a particular endpoint or dependency. I would correlate application metrics, traces, infrastructure metrics, database performance, and recent changes. For PostgreSQL, I would investigate slow queries, locks, connection saturation, and resource utilization. If customers are significantly affected, I would prioritize safe mitigation, then conduct root cause analysis after service stabilizes.

24. What is your approach to incident management?

Sample answer

I assess severity, confirm impact, establish an incident owner, and communicate with stakeholders. I use monitoring, logs, traces, and recent deployment history to narrow down the cause. The first objective is to restore service safely, possibly by rolling back a change or applying a workaround. After recovery, I document the timeline, root cause, contributing factors, and preventive actions.

25. What is a blameless postmortem?

Sample answer

A blameless postmortem is a structured review focused on system conditions, contributing factors, and improvements rather than personal blame. It documents what happened, customer impact, detection and response times, root cause, and actions needed to prevent recurrence. Actions should have clear owners and priorities.

5. Multi-tenant SaaS architecture and security

26. What is a multi-tenant SaaS application?

Sample answer

A multi-tenant SaaS application serves multiple customers using a shared platform. Tenants may share application instances, databases, schemas, or infrastructure while their data and access remain logically or physically isolated. The design must balance security, scalability, operations, and cost. I would verify tenant isolation in authentication, authorization, API requests, database queries, logs, caches, and background jobs. I would also monitor tenant-specific usage and noisy-neighbor effects.

27. How would you prevent one tenant from accessing another tenant’s data?

Sample answer

I would implement tenant-aware authentication and authorization, enforce tenant context server-side, and never trust a client-supplied tenant ID without validation. At the database layer, I would consider PostgreSQL row-level security where appropriate. I would apply least privilege and test cross-tenant access scenarios, then monitor suspicious access patterns and check that caches, jobs, and logs do not expose tenant data.

28. How do you secure a cloud-native application?

Sample answer

I would apply defense in depth: least-privilege identity and access management, network segmentation, encryption in transit and at rest, secure secret management, dependency and container scanning, and controlled CI/CD permissions. I would patch supported software, restrict database network access, audit privileged actions, and monitor suspicious activity. Security should be integrated into development and deployment rather than treated as a final check.

6. Scenario-based interview questions

Scenario 1: Production is down after a deployment. What do you do?

Sample answer

My first priority is restoring service and limiting customer impact. I confirm the incident, affected services, and scope. I check alerts, deployment history, application health, and error rates. If the deployment is the likely cause and rollback is safe, I initiate the established rollback. I verify recovery through health checks, logs, and customer-facing metrics, communicate status, then investigate root cause and add preventive actions such as stronger smoke tests, canary checks, or deployment gates.

Scenario 2: PostgreSQL CPU utilization suddenly reaches 95%.

Sample answer

I assess whether the database is responding and whether application latency or errors are increasing. I inspect active queries, execution plans, connections, locks, database load, and recent changes. I look for inefficient queries, missing or ineffective indexes, high traffic, and connection-pool issues. If a query causes severe impact, I work with its owner to mitigate it safely rather than terminating sessions indiscriminately. After stabilizing, I tune the workload and add early-warning alerts.

Scenario 3: The primary database becomes unavailable.

Sample answer

I confirm the failure and check database health, connectivity, and provider alerts. If a supported HA failover is configured, I follow the established procedure and verify the standby becomes active. I validate application connectivity, transaction behavior, and data consistency. If automatic failover fails, I follow the recovery plan and involve the database owner or provider. After recovery, I investigate the cause and verify backups, replication, and failover mechanisms.

Scenario 4: A deployment works in staging but fails in production.

Sample answer

I compare application version, configuration, identity permissions, secrets, network rules, resource limits, dependency versions, and database schema. I inspect logs and events to identify the first failing component. If customer impact is significant, I use the safest rollback or mitigation. Then I correct the environment difference and improve configuration validation, automated testing, or deployment parity.

Scenario 5: One tenant causes high resource consumption and affects others.

Sample answer

I identify the tenant and workload responsible using metrics and telemetry, while confirming impact on other tenants. I apply suitable rate limits, quotas, workload controls, or targeted scaling. I investigate expensive queries, background jobs, or abnormal request patterns. Longer term, I establish per-tenant monitoring, capacity thresholds, and safeguards against noisy-neighbor effects without compromising isolation.

Scenario 6: A PostgreSQL migration fails during a production release.

Sample answer

I determine whether the migration partially completed and whether the application can safely operate with the current schema. I avoid blindly rerunning or reverting it because it may not be safely reversible. I assess data integrity, transaction status, and application compatibility, then follow the recovery plan. For future releases, I prefer backward-compatible migrations: add schema first, deploy compatible code, migrate data safely, and remove old schema elements in a later release.

Scenario 7: Your monitoring system generates too many alerts.

Sample answer

I review which alerts are actionable, noisy, or merely symptoms. I tune thresholds against baselines and service objectives, aggregate and deduplicate events, and distinguish urgent customer-impacting issues from informational events. Alerts should have clear owners and runbook links. The goal is to surface important incidents quickly without overwhelming the team.

7. PostgreSQL fundamentals and basics

PostgreSQL is an open-source relational database management system (RDBMS). It stores structured data in tables and supports SQL, transactions, constraints, indexes, joins, JSONB, and row-level security.

Key terms

- **Database:** a collection of database objects.
- **Table:** data organized into rows and columns.
- **Row:** one record; **column:** a field with a defined type.
- **Primary key:** uniquely identifies a row.
- **Foreign key:** enforces a relationship with another table.
- **Index:** can speed up certain data retrieval operations.
- **Transaction:** operations handled as a unit.
- **Schema:** a namespace for organizing database objects.
- **Query:** an instruction to retrieve or manipulate data.

7.1 Create a table

```
CREATE TABLE employees (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    department VARCHAR(50),
    salary NUMERIC(12, 2),
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

IDENTITY generates an ID; PRIMARY KEY enforces uniqueness and non-null values; NUMERIC is suitable for exact decimal values; TIMESTAMPTZ stores a timestamp with time-zone-aware semantics; DEFAULT NOW() supplies a default timestamp when omitted.

7.2 Insert, select, filter, sort, and limit

```
INSERT INTO employees (name, department, salary)
VALUES
    ('Alice', 'DevOps', 85000.00),
    ('Bob', 'Engineering', 90000.00),
    ('Charlie', 'DevOps', 95000.00);

SELECT * FROM employees;

SELECT name, salary
FROM employees
WHERE department = 'DevOps';

SELECT name, salary
FROM employees
ORDER BY salary DESC;

SELECT *
FROM employees
LIMIT 10;
```

WHERE filters rows, ORDER BY sorts results, and LIMIT restricts the number of returned rows.

7.3 Update and delete

```
UPDATE employees
SET salary = 100000.00
WHERE id = 1;

DELETE FROM employees
WHERE id = 1;
```

Always verify the WHERE clause before running an update or delete in a real environment. Without WHERE, all rows may be affected.

7.4 Common filters

```
SELECT * FROM employees WHERE salary > 90000;
SELECT * FROM employees WHERE salary BETWEEN 80000 AND 95000;
SELECT * FROM employees WHERE department IN ('DevOps', 'Engineering');
SELECT * FROM employees WHERE name LIKE 'A%';
SELECT * FROM employees WHERE department IS NULL;
```

Use IS NULL, not = NULL.

7.5 SQL joins

```
CREATE TABLE departments (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    department_name VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE staff (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    department_id BIGINT REFERENCES departments(id)
);

SELECT s.name, d.department_name
FROM staff s
INNER JOIN departments d
  ON s.department_id = d.id;

SELECT s.name, d.department_name
FROM staff s
LEFT JOIN departments d
  ON s.department_id = d.id;
```

Join	What it returns
INNER JOIN	Rows with matching values in both tables.
LEFT JOIN	All left-table rows plus matching right-table rows; unmatched right values are NULL.
RIGHT JOIN	All right-table rows plus matching left-table rows.
FULL OUTER JOIN	All rows from both sides, matching where possible.

7.6 Aggregate functions and grouping

```
SELECT COUNT(*) FROM employees;
SELECT AVG(salary) FROM employees;
SELECT MAX(salary) FROM employees;
SELECT SUM(salary) FROM employees;

SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department;

SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 1;
```

WHERE filters individual rows before grouping; HAVING filters groups after aggregation.

7.7 Indexes and query performance

```
CREATE INDEX idx_employees_department
ON employees (department);

EXPLAIN
SELECT *
FROM employees
WHERE department = 'DevOps';

EXPLAIN ANALYZE
SELECT *
FROM employees
WHERE department = 'DevOps';
```

What is an index?

Sample answer

An index can help PostgreSQL locate rows without scanning the whole table. Indexes are useful for columns frequently filtered, joined, or sorted, depending on query shape and data distribution. They consume storage and add write overhead, so more indexes are not always better.

What is an execution plan?

Sample answer

EXPLAIN shows the plan the optimizer expects to use. EXPLAIN ANALYZE executes the query and reports actual execution information. Use EXPLAIN ANALYZE carefully in production because it executes the statement; data-changing statements can have real side effects.

How do you troubleshoot a slow PostgreSQL query?

Sample answer

Identify the slow query through monitoring or query statistics, examine its execution plan, and look for inefficient scans or joins, missing or ineffective indexes, inaccurate row estimates, excessive data retrieval, and lock waits. Check resource utilization and connection saturation too. After changes, compare execution time and resource use to verify improvement.

What is the difference between a sequential scan and an index scan?

Sample answer

A sequential scan reads table rows directly and can be efficient when many rows are needed. An index scan uses an index to locate matching rows and may then fetch table rows. A bitmap scan can collect row locations in batches. A sequential scan is not automatically bad; PostgreSQL may correctly choose it when a large portion of a table must be read.

What is VACUUM?

Sample answer

PostgreSQL uses MVCC, so updates and deletes can leave obsolete row versions. VACUUM helps clean up dead tuples and make space reusable. VACUUM ANALYZE also updates planner statistics. Routine autovacuum should generally remain enabled and appropriately configured. VACUUM FULL rewrites the table and requires stronger locking, so it should not be used casually in production.

7.8 Transactions and ACID

```
BEGIN;

UPDATE accounts
SET balance = balance - 100
WHERE id = 1;

UPDATE accounts
SET balance = balance + 100
WHERE id = 2;

COMMIT;
```

This example assumes valid account IDs and appropriate business-rule checks. If an operation fails, use ROLLBACK rather than committing, provided the transaction remains active.

Property	Meaning
Atomicity	Operations in a transaction succeed together or are rolled back.
Consistency	Transactions preserve defined integrity rules.
Isolation	Concurrent transactions are controlled according to the isolation level.
Durability	Committed changes are preserved according to durability guarantees.

What are locks and deadlocks?

Sample answer

Locks coordinate concurrent operations. A deadlock occurs when transactions wait on resources held by one another in a cycle. PostgreSQL detects deadlocks and aborts one transaction. To reduce deadlocks, keep transactions short, acquire resources in a consistent order, optimize queries, and investigate lock waits instead of restarting the database.

7.9 Backup, restore, PITR, replication, and pooling

```
# Logical backup in custom format
pg_dump -h <host> -U <user> -d <database> -F c -f backup.dump

# Restore into an existing target database
pg_restore -h <host> -U <user> -d <target_database> backup.dump

# Check whether a PostgreSQL server is accepting connections
pg_isready
```

These commands assume the target database exists and the account has the required permissions. Avoid placing passwords in shell history or command arguments.

How do you back up PostgreSQL?

Sample answer

A logical backup can be made with pg_dump and restored with pg_restore. For production, I would also evaluate managed-service backups, point-in-time recovery, and physical backup strategies where appropriate. A backup is only useful if it can be restored successfully.

What is point-in-time recovery (PITR)?

Sample answer

PITR restores a database to a particular point in time within its supported recovery window, typically using a base backup and archived WAL records. For Azure Database for PostgreSQL, I would verify the configured retention, available restore targets, regional options, and documented service limitations.

What is replication?

Sample answer

Replication maintains copies of database changes on another server or instance. Physical replication uses database-level changes via WAL; logical replication uses logical decoding and publication/subscription mechanisms to replicate selected data changes. Replication can support availability and read scaling but does not replace backups.

What is connection pooling?

Sample answer

Connection pooling reuses database connections instead of repeatedly opening new ones. It reduces connection overhead and can protect PostgreSQL from too many concurrent connections. Pool sizes should be selected carefully based on database capacity and application concurrency.

What is row-level security (RLS)?

Sample answer

RLS allows policies to control which rows a database role can access or modify. In a multi-tenant SaaS platform, it can add a tenant isolation layer. It must be configured carefully, considering policies, application roles, table ownership, and privileged access paths.

7.10 PostgreSQL commands to memorize

Command	Purpose
\\l	List databases in psql
\\c database_name	Connect to a database
\\dt	List tables in the current search path
\\d table_name	Describe a table
SELECT version();	Display PostgreSQL version
SELECT current_database();	Show current database
SELECT NOW();	Return the current transaction timestamp
EXPLAIN ANALYZE ...	Execute and inspect a query plan
pg_dump	Create a logical backup
pg_restore	Restore a supported pg_dump archive
pg_isready	Check server connection readiness

8. Employer application questions

1. Which of the following statements best describes your right to work in the Philippines?

Sample answer

Choose the option that reflects your actual legal status. If you are a Filipino citizen and legally entitled to work in the Philippines, select the closest option to: “I am a Filipino citizen and have the right to work in the Philippines without sponsorship.” The exact wording depends on the form. Do not select this if it does not reflect your status.

2. What’s your expected monthly basic salary?

Suggested initial negotiation target: PHP 120,000–PHP 140,000 gross monthly basic salary. This is a suggested starting range, not a verified market rate.

Suggested application wording:

My expected monthly basic salary is in the range of PHP 120,000 to PHP 140,000, depending on the overall compensation package, role responsibilities, and benefits. I am open to discussing the range based on the opportunity and expectations of the position.

If the form accepts only one number, PHP 130,000 could be an initial target if it aligns with your personal minimum and current compensation. Consider salary band, on-call responsibilities, shift requirements, and benefits.

3. How many years’ experience do you have as a DevOps Engineer?

Sample answer

Enter 6 years, based on the experience previously provided. If the form uses a dropdown, choose the band that includes six years. Count professional experience accurately and do not double-count overlapping jobs.

4. Which of the following types of qualifications do you have?

Sample answer

Choose the option matching your highest completed qualification. Examples include a relevant bachelor’s degree, diploma or vocational qualification, or equivalent practical experience if offered and applicable. Confirm your actual credentials before selecting.

5. How many years of Site Reliability Engineering experience do you have?

Sample answer

Enter the duration that accurately reflects your dedicated SRE experience and/or relevant reliability responsibilities according to the employer’s definition. SRE duties can overlap with DevOps, but do not automatically enter six years just because you have six years in DevOps. Be ready to explain the reliability work you actually performed.

If asked to clarify the difference

My primary title has been DevOps Engineer, and my experience includes reliability-related responsibilities such as automation, monitoring, troubleshooting, deployment safety, and operational support. I understand that SRE places additional emphasis on measurable reliability objectives, incident management, and reducing operational toil. I’m interested in applying and expanding those practices in this role.

9. Behavioral interview questions

1. Why should we hire you?

Sample answer

You should consider me because I bring six years of DevOps experience and a strong foundation in cloud infrastructure, automation, CI/CD, and secure software delivery. I understand that this role requires more than deploying applications: it requires reliable infrastructure, reduced deployment risk, effective incident response, and continuous operational improvement. My AWS background, together with my Azure and Azure DevOps experience, gives me a solid foundation to contribute. I am prepared to deepen my Azure platform and PostgreSQL skills to meet the needs of your environment.

2. What is your biggest weakness?

Sample answer

My primary cloud experience is in AWS, so I am continuing to strengthen my depth in Azure-specific architecture and managed services. I already have experience with Azure Cloud and Azure DevOps, and I'm building on that foundation by studying Azure networking, high availability, monitoring, and managed database operations. I approach learning systematically by understanding the architecture, practicing in a controlled environment, and validating what I learn against team standards.

Interview tip: Use this only if it accurately reflects your current skill profile.

3. Tell me about a challenging technical problem you solved.

Sample answer

Use the STAR method. Situation: describe the production or project problem. Task: explain what you were responsible for. Action: describe the troubleshooting steps you personally took. Result: explain the verified outcome. Example template: "We encountered [specific issue], affecting [service or workflow]. I reviewed [logs, pipeline output, metrics, or configuration], identified [actual root cause], and implemented [corrective action]. After validating the fix, I added or recommended [preventive control]. The result was [actual outcome]." Replace every bracketed section with a real example.

4. How do you prioritize multiple incidents and tasks?

Sample answer

I prioritize by customer impact, severity, security risk, and business criticality. A production outage or potential data exposure takes precedence over routine optimization. I communicate priorities, involve the right owners, and track remaining work. For recurring issues, I look for automation or process improvements that reduce future incidents.

5. Where do you see yourself in three to five years?

Sample answer

I want to grow into a senior platform or reliability engineering role where I contribute to cloud architecture, automation strategy, and production reliability. I want to deepen my expertise in Azure, Kubernetes, PostgreSQL, observability, and disaster recovery while improving my ability to design secure, scalable, resilient systems. I would also like to contribute to engineering standards and help other team members improve delivery and operations.

6. What are your salary expectations, and are you negotiable?

Sample answer

Based on the scope of the role and my six years of DevOps experience, I'm targeting a gross monthly basic salary in the range of PHP 120,000 to PHP 140,000. However, I'm open to discussing the complete compensation package, including benefits, responsibilities, and growth opportunities.

7. Are you comfortable supporting production incidents outside normal working hours?

Sample answer

Answer based on your real availability and preferences. If open to an agreed on-call rotation: “I understand that a business-critical SaaS platform requires incident response outside standard hours when necessary. I’m open to discussing the on-call rotation, escalation expectations, and how incident coverage is organized.” If you are only available for day or midshift work, clarify whether after-hours support is occasional compensated on-call or a regular schedule requirement.

10. Questions to ask the interviewer

1. What does your current Azure architecture look like?

Could you share an overview of your Azure infrastructure and how application services, PostgreSQL databases, and containerized workloads are deployed?

2. What are your main PostgreSQL challenges?

Is the primary focus routine database operations, performance optimization, high availability, or disaster recovery?

3. How do you measure platform reliability?

Do you have established SLOs, RTO/RPO targets, and regular disaster recovery exercises?

4. What is your current deployment process?

Are you using GitHub Actions, Azure DevOps, or another CI/CD platform, and what improvements would you like this person to deliver?

5. What are the on-call and working-hour expectations?

Could you explain the incident escalation process, support rotation, and normal working schedule for this position?

11. Seven-day study plan

Day	Focus	Tasks
1	Azure architecture and HA	Azure networking, identity, Key Vault, Monitor; zones, redundancy, failover; explain an HA SaaS design.
2	PostgreSQL fundamentals	SQL CRUD, joins, aggregates, keys, indexes, transactions, ACID; write 10–15 queries.
3	PostgreSQL operations	EXPLAIN ANALYZE, indexes, locks, pooling, pg_dump/restore, PITR, Azure PostgreSQL HA.
4	CI/CD and IaC	GitHub Actions, approvals, Terraform modules/state/plans, rolling, blue-green, canary.
5	Containers and observability	Docker, Kubernetes pods/deployments/services/probes, metrics, logs, traces, SLOs.
6	Incident response and DR	Practice outage and database-failure scenarios; explain RTO/RPO; prepare two STAR examples.
7	Mock interview	Practice introduction and screening answers; answer technical questions aloud; review gaps.

Final interview advice

- PostgreSQL production troubleshooting: indexes, execution plans, transactions, connection limits, backups, and recovery.
- Azure high availability and disaster recovery: zones, managed database HA, RTO, RPO, and recovery testing.
- GitHub Actions and Terraform: secure credentials, approvals, state management, and safe deployments.
- SRE principles: SLOs, actionable alerts, incident response, and blameless postmortems.
- Multi-tenant SaaS security: tenant isolation, least privilege, capacity management, and noisy-neighbor protection.

If hands-on PostgreSQL experience is limited, do not present yourself as a database administrator. Demonstrate that you understand the fundamentals, can investigate problems methodically, and can support database operations safely alongside the development and database teams.